

NAME: S.GAYATHERI

REG NO:192521306

COURSE CODE: CSA0619

COURSE NAME : Design

Analysis of Algorithm

CO4 – AT2 – Industry Problem Solving Task

Q12. Logistics Route Optimization

Real-World Problem

A logistics company needs to deliver goods to 18 different cities, visiting each city exactly once and returning to the original warehouse or distribution center. The objective is to minimize the total transportation cost, which may include:

- Fuel cost
- Travel distance
- Travel time
- Toll charges
- Vehicle operating cost

This problem can be modelled as the Travelling Salesman Problem (TSP).

Industry Constraints

In a real logistics environment, the problem is affected by several constraints:

1. Distance or cost between cities may be different in each direction.
2. Traffic conditions can change travel time.
3. Delivery deadlines may impose time constraints.
4. Road closures or weather conditions may change routes.
5. Fuel and vehicle operating costs must be minimized.
6. Real logistics companies may operate multiple vehicles rather than one vehicle.

Therefore, the 18-city TSP represents a simplified but useful optimization model for route planning.

Graph Representation

The logistics network can be represented as a weighted graph:

$$G = (V, E)$$

Where:

- V represents the 18 cities.
- E represents routes between cities.
- Each edge has a weight representing distance or transportation cost.

The objective is to find the minimum-cost Hamiltonian cycle.

Dynamic Programming Approach: Held–Karp Algorithm

The Held–Karp algorithm solves TSP using:

- Dynamic Programming
- Bitmasking
- Graph theory
- Optimization techniques

State Representation

The DP state is:

$$DP[mask][i]$$

Where:

- mask represents the set of visited cities using binary bits.
- i represents the current city.
- $DP[mask][i]$ represents the minimum cost to start from the origin, visit all cities in mask, and end at city i.

For example, for 5 cities:

$$mask = 10111$$

means the cities corresponding to the 1 bits have already been visited.

Recurrence Relation

To reach city i , the algorithm considers every possible previously visited city j .

$$DP[mask][i] = \min_{j \neq i} [DP[mask \setminus \{i\}][j] + Cost[j][i]]$$

This ensures that all possible routes are considered while avoiding repeated computation.

Base Case

If the logistics vehicle starts at city 0:

$$DP[1][0] = 0$$

Final Cost

After visiting all cities:

$$Minimum\ Cost = \min_i [DP[(1 \ll n) - 1][i] + Cost[i][0]]$$

This includes the cost of returning to the original warehouse.

Proposed System Design

Distance / Cost Data



Data Validation



Distance Matrix



TSP Held–Karp Algorithm



Dynamic Programming Table



Optimal Route Calculation



Minimum Cost Route



Logistics Dashboard / Report

Solution Workflow

Step 1: Collect Logistics Data

Collect information about:

- City locations
- Distance between cities
- Fuel costs
- Toll costs
- Transportation time

Step 2: Build the Distance Matrix

For 18 cities:

$$Distance[18][18]$$

The matrix contains:

$$18 \times 18 = 324$$

possible cost values.

Step 3: Initialize the DP Table

The initial state is:

$$DP[1][0] = 0$$

All other states are initialized to infinity.

Step 4: Generate Subsets

Use bitmasking to generate different combinations of visited cities.

The number of subsets for 18 cities is:

$$2^{18} = 262,144$$

Step 5: Calculate Minimum Cost

For each subset and current city, calculate the minimum possible route cost using the recurrence relation.

Step 6: Return to Origin

Add the cost of returning from the final city to the starting warehouse.

Step 7: Reconstruct the Route

Store the previous city for each DP state to reconstruct the optimal route.

Use of Modern Tools

A practical implementation can use the following tools.

Tool	Purpose
Python	Algorithm implementation and testing
NumPy	Distance matrix processing
Google OR-Tools	Industrial route optimization
NetworkX	Graph modelling and visualization
Gurobi Optimizer	Advanced optimization
GIS / Map APIs	Real-world road and location data
Cloud Computing	Large-scale route processing

Recommended Implementation

For an academic project:

Python + NumPy + Held–Karp Algorithm

For a real industry system:

Python/C++ + GIS data + Google OR-Tools + Cloud Infrastructure

This combination provides both algorithmic accuracy and real-world scalability.

Performance Analysis

Time Complexity

The Held–Karp algorithm has:

$$O(n^2 2^n)$$

For 18 cities:

$$\begin{aligned} 18^2 \times 2^{18} \\ 324 \times 262,144 \approx 84,934,656 \end{aligned}$$

Therefore, the algorithm performs approximately 85 million computational operations in the worst-case order, making it feasible for 18 cities on modern hardware.

Space Complexity

The space complexity is:

$$O(n 2^n)$$

For 18 cities:

$$18 \times 2^{18} = 4,718,592$$

DP states are required.

This shows that memory usage becomes an important limitation as the number of cities increases.

Feasibility in Real-World Logistics

For 18 cities, Held–Karp is feasible because:

- The number of DP states is manageable.
- Modern computers can process millions of operations efficiently.
- It produces an exact optimal solution.

However, real logistics systems may contain:

- Hundreds of delivery locations.
- Multiple vehicles.
- Dynamic traffic.
- Delivery time windows.
- Vehicle capacity constraints.

For these situations, an exact TSP algorithm becomes computationally expensive.

The company should therefore use a hybrid approach:

Source Code

[illegible]

Output

```
Output

Optimal Route:
0 -> 2 -> 3 -> 1 -> 0

Minimum Transportation Cost:
80

=== Code Execution Successful ===
```

Final Conclusion

The logistics route optimization problem can be effectively modelled as a Travelling Salesman Problem. For 18 cities, the Held–Karp Dynamic Programming algorithm provides an exact minimum-cost route.

The state representation is:

$$DP[mask][i]$$

and the recurrence relation calculates the minimum cost by considering all possible previous cities.

The main strengths are:

- Guaranteed optimal solution
- Suitable for small and medium-sized logistics networks
- Systematic Dynamic Programming approach

The main limitation is exponential complexity:

$$Time = O(n^2 2^n)$$

$$Space = O(n 2^n)$$

Therefore, 18 cities is feasible, but as the number of cities increases, the algorithm becomes difficult to scale. A real-world logistics company should use Held–Karp for smaller optimization problems and modern heuristic or VRP tools for large-scale, dynamic logistics systems.